

Best practices of Nodejs (how to scale an application and how to prevent API failure)

Firstly Nodejs is a runtime environment of javascript which allows to execute javascript code outside browser. Nodejs heavily dependent on non-blocking input / output. Usually in tradition servers the backend was built using PHP/Java + Apache which leads to handling user request by creating multiple threads means when multiple user send requests then server handles those requests by creating multiple threads. It leads to more utilization of the resources. Whereas incase of Nodejs when user sends multiple requests then only one single thread is used to handle multiple requests without the use of any other thread. What it does it sends small tasks like database queries, network requests and file operation to the OS(operating system) and continues handling other requests, when the task finishes the result comes back to Nodejs and it handles it. But the problem here is when there is a task which takes much time of the OS to complete so it remains there in the main thread resulting the main thread get's stuck and nodejs server gets frozen. So the main thing is nodejs performs very well when each piece of work is small and doesn't blocks the event loop.

Engineering for Resiliency

1. You should not block the event loop, keep API request handling code lightweight and fast. Avoid synchronous and blocking operations which takes large amount of time to execute through main thread.
2. Move heavy work and time taking work to background workers so that it doesnot take time of other user and the task is waiting inside the API request.
3. Async and await function helps to read and understand asynchronous code easily when an asynchronous operation fails then we put them in try or catch block to handle the unexpected problems.
4. There are many Nodejs operations which returns promises and a promise can either lead to failure or success. If there is a promise rejection and we donot handle it using try/catch block then it becomes unhandles rejection which leads to system failure and cause problem.
5. There must be a centralized error handling system there should not be different routing API routes to handles errors there must be a central middleware which handles all the errors.
6. There must be a proper HTTP status code which tells users about the client what happened like 200 means success, 400 means bad input, 401 means authentication is

required, 404 means the resource wasn't found, and 500 means a server-side error occurred.

7. There must be a global error handler which activates when any error handling is missed by the catch/try block so nodejs gets it and activate the global error handler. A global error handler is like a **backup safety net**. It shutdowns the application so that process manager can restart it.
8. There must be strict input validation like to check the data sent by the user before calling the API if we need text and user sends numbers so it might lead to an error.
9. Tools like **Zod** or **Joi** let you define what the input should look like.

*The goal is to **handle failures safely**, return a useful response to the user, clean up resources when necessary, and keep the application available whenever possible.

Optimizing Node.js Performance (Writing High-Throughput Code)

1. Establishing a new network connection to a database like PostgreSQL or MongoDB is a highly expensive operation. If your API attempts to open and close a fresh connection for every single HTTP request under high traffic, the latency will skyrocket, and the database will eventually reject connections due to exhaustion. **Connection Pooling** keeps several connections ready to use. When a request comes, Node.js takes one connection, performs the database query, and **returns it to the pool** so another request can reuse it.
2. We must initialize in memory caching, which stores frequently used data in the cache so that there is no need to go to database everytime we need something like :- if 1,000 users request the same product list, instead of asking the database 1,000 times, we can store the result in **Redis**.

Architectural Scaling (Handling Massive Concurrent Traffic)

1. When one Nodejs server reaches more users that it is not enough for a single server to handles so many users so therefore architectural scaling is really important. It is done with 2 ways:-
 1. Vertical scaling : making existing server more powerful by adding more CPU, RAM and hardwares.
 2. Horizontal scaling: eventually one powerful server won't be enough we can add multiple nodejs servers and a load balancer which balances the load on CPU and distributes the users accordingly.
 3. Stateless architecture means **not storing important user information only inside one server**. If a user's next request goes to another server, that server should still be able to recognize the user. So sessions can be stored in a shared system like **Redis**, or authentication can use **JWT**.